# Efficient Die-to-Die Communication: UCIe Link Simulation and Optimization in a Chiplet-Based System

Kunyue Li, Shuaipeng Li, Xiaoyan Li, Zizheng Dong, Sai Gao, Jialei Sun, Naifeng Jing, *Senior Member, IEEE*, Qin Wang, *Member, IEEE*, Guanghui He, *Member, IEEE*, and Jianfei Jiang, *Member, IEEE*

***Abstract*—With the increasing complexity of chiplet-based architectures and the growing adoption of heterogeneous computing, efficient and high-precision simulation frameworks are essential for evaluating chiplet interconnect performance. This paper presents an implementation of the UCIe protocol (Universal Chiplet Interconnect Express) within the gem5 simulation environment, providing a comprehensive system-level modeling framework for UCIe-based interconnects. The proposed UCIe link model supports full-stack protocol simulation, including transaction-level processing, die-to-die adaptation, and physical layer interactions. A flit-based packing mechanism is introduced to enable accurate transmission modeling, and an Ack/Nak-based retry mechanism is introduced to ensure robust data integrity. An optimized event-driven scheduling strategy is incorporated to address performance bottlenecks observed in traditional UCIe simulations. Experimental evaluations validate the accuracy and efficiency of the proposed UCIe link model by comparing its latency with theoretical values under different computational workloads. The results show that the model closely matches the theoretical predictions, with a latency deviation of less than 0.5% in most cases. Additionally, performance comparisons between UCIe interconnect, PCIe interconnect, and direct memory transfer reveal that UCIe incurs only a minor protocol overhead (less than 0.7%), making it a practical and scalable solution for multi-chipset interconnects. The proposed UCIe simulation framework provides a high-precision virtual verification platform for chiplet system design, enabling low-cost, high-accuracy performance evaluations, and lays the foundation for future optimizations in large-scale chiplet-based architectures.**

***Index Terms*—UCIe, PCIe, chiplet, gem5, interconnect.**

## I. Introduction

With the increasing manufacturing costs of chips under advanced processes and the slowing down of Moore's Law, chiplet technology has emerged as a crucial pathway to sustain improvements in semiconductor performance. By decomposing a monolithic system-on-chip (SoC) into multiple modular chips and heterogeneously integrating them, this technology significantly reduces manufacturing costs while enhancing production yield [1], [2]. Using advances in 2.5D and 3D advanced packaging technologies, chiplet technology has substantially improved the stability of die-to-die interconnections over the past decade. This technology demonstrates considerable potential and practical value in modern chip design, whether by assembling multiple computational array chiplets into a multi-core CPU or integrating various functional chiplets into a complete chip.

However, fragmented interconnect standards within the chiplet ecosystem have significantly constrained its technological potential. To address this challenge, Intel, AMD, ARM, and eight other leading companies jointly introduced the open standard Universal Chiplet Interconnect Express (UCIe 1.0) in 2022 [3]. UCIe inherits the essence of Advanced Interface Bus (AIB) technology [4], defining electrical signaling standards and specifying physical channel requirements for die-to-die communication at the physical layer. Additionally, it provides several essential functionalities at the adapter and protocol layers, including link state management and compatibility with higher-level protocols. Similarly to the role of Peripheral Component Interconnect Express (PCIe) in the expansion card ecosystem, UCIe establishes clear guidelines for collaborative methodologies in multi-chiplet systems, making it a pivotal technology for building next-generation high-performance computing (HPC) systems.

In architecture research, the gem5 simulator has become the gold standard tool for chip design verification due to its full system simulation capabilities and cycle precision [5], [6]. However, the current gem5 framework lacks support for the UCIe protocol, leading to two critical challenges: (1) Researchers are unable to evaluate the system-level performance impact of UCIe interconnects in a virtualized environment; (2) Hardware development teams still rely on physical prototype verification, significantly increasing both time and financial costs.

To address these challenges, this paper proposes an implementation framework for UCIe protocol simulation within the gem5 environment, designing a comprehensive system-level simulation framework. By constructing a configurable die-to-die (D2D) interface model and employing a layered verification methodology, we achieve full-stack protocol

Received 17 March 2025; revised 11 May 2025; accepted 15 July 2025. Date of publication 22 July 2025; date of current version 15 December 2025. This work was supported by the National Key Research and Development Program of China under Grant 2023YFB4403402. This article was recommended by Guest Editor Q. Hao. *(Kunyue Li and Shuaipeng Li contributed equally to this work.) (Corresponding author: Jianfei Jiang.)*

The authors are with the State Key Laboratory of Micro-Nano Engineering Science, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: 2360482543@sjtu.edu.cn; sjtuer_lsp@sjtu.edu.cn; jiangjianfei@sjtu.edu.cn).

Digital Object Identifier 10.1109/JETCAS.2025.3590822

2156-3357 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

simulation from the transaction to signal levels. Experimental results demonstrate that this approach significantly enhances the efficiency of interconnect architecture verification. It not only fills a critical gap in open-source simulation toolchains but also provides a high-precision virtual verification platform for chiplet system design.

The major contributions of our work can be summarized as:

1) Proposed a UCIe link protocol model on gem5 based on the flit packing and retry mechanisms, providing a simulation framework to guide future chiplet design.
2) Validated the latency accuracy of the UCIe link model, demonstrating that the measured latency deviates from the theoretical latency by only 0.5%.
3) Compared the system performance of UCIe, PCIe, and direct data transfer in a general chiplet application scenario, showing that the proposed UCIe link model exhibits only a 0.7% performance difference compared to direct transfer.

The remainder of this paper is organized as follows: Section II discusses prior work on this topic. Section III presents the proposed UCIe link model, focusing on its flit packing and retry mechanisms. Section IV conducts experiments and analyses on the link model's latency and performance. Finally, Section V concludes the paper.

## II. Related Work and Novel Contributions

Since UCIe 1.0 was proposed in 2022, it has received widespread industry support. However, existing research on UCIe remains limited, with most studies focusing on physical-layer implementations, while system-level UCIe link simulation based on gem5 is mainly unexplored. Several notable studies have investigated UCIe interconnects from different perspectives: In terms of physical-layer implementations, Li et al. designed a UCIe channel in Grounded Shielding (GS) mode using a 5Mi Si-interposer process [7]. Shimomura et al. proposed a single-layer routing layout method for interchip connections to reduce UCIe interposer costs [8]. Kim et al. employed peak distortion analysis to determine the worst-case bit sequence for a rectangular eye diagram mask, thereby optimizing the UCIe interface channel design [9].

Regarding protocol modeling, Alian et al. conducted gem5-based modeling of PCIe, analyzing the impact of PCIe bandwidth on system performance [10]. Medina et al. simulated full-system gem5 modeling in wireless routing and investigated optimization strategies in system-level simulations [11]. Jose and Shankar sexplored UCIe protocol simulation on VisualSim Architect, examining system-level optimization strategies enabled by UCIe simulation [12]. Zheng et al. introduced a chiplet-based many-core architecture, incorporating adaptive routers and interconnects [13]. Li et al. proposed a basic UCIe protocol simulation framework, conducting preliminary model validation and latency analysis [14]. However, in most studies, physical-layer latency simulation is often based on empirical values or fixed estimates. Additionally, many studies treat line delay strictly, focusing on specific packaging and transmission media. Few consider the additional SerDes delay at the physical layer, and the protocol layer is often neglected, leading to models that lack generality and representativeness.

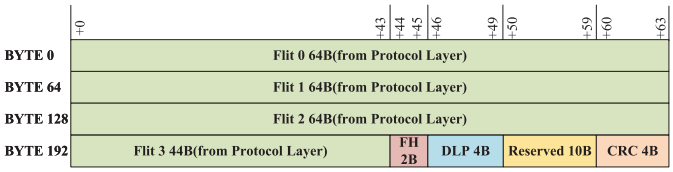



Fig. 1. Layout of Standard 256B Flit Format.

This work implements a UCIe link model in gem5 to address these issues and comprehensively evaluates UCIe protocol performance in a CPU-accelerator application scenario. This approach effectively resolves the challenges mentioned above and provides a high-precision virtual verification platform for chiplet system design.

## III. UCIe Link Model Implementation

This work simulates the UCIe link standard based on PCIe 6.0, which adopts the standard 256B flit mode defined in the Universal Chiplet Interconnect Express (UCIe) Specification Revision 1.0 [3] and the PCI Express Base Specification Revision 6.0.1 [15].

Figure 1 illustrates the layout of a standard 256B flit-format data packet. This packet comprises 236B from the PCIe protocol layer, supplemented by a 6B data link packet (DLP) from the UCIe die-to-die adapter and a 4B cyclic redundancy check (CRC).

UCIe1.0 permits two different package options. The standard package option is characterized by a more significant bump pitch and a broader channel extension range. This configuration inherently limits the number of die-to-die channels accommodated within the same physical footprint [16]. In the mainband of the interface, the standard package option provides only one-fourth of the data channels compared to advanced package option. Specifically, standard package option includes 16 data channels on the transmitting side per module, whereas advanced package option supports 64 data channels.

Both package options are compatible with data rates of 4, 8, 12, 16, 23, and 32 GT/s. This model selects the most fundamental electrical layer configuration of standard package interconnect, featuring a 16-bit data width and a 4 GT/s data rate, resulting in a PHY operating speed of 64 GT/s. The digital data path is configured to operate at 250 MHz with a 256-bit width to maintain synchronization between the digital section and the analog PHY. This setup ensures compatibility and synchronization between the digital and analog PHY components.

### A. Overview of UCIe Link Model

Gem5 facilitates object communication through packet transmission across ports. There are two types of ports in gem5: primary ports and secondary ports. The connection between these two port types is unidirectional, where the primary port is responsible for sending requests and receiving

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

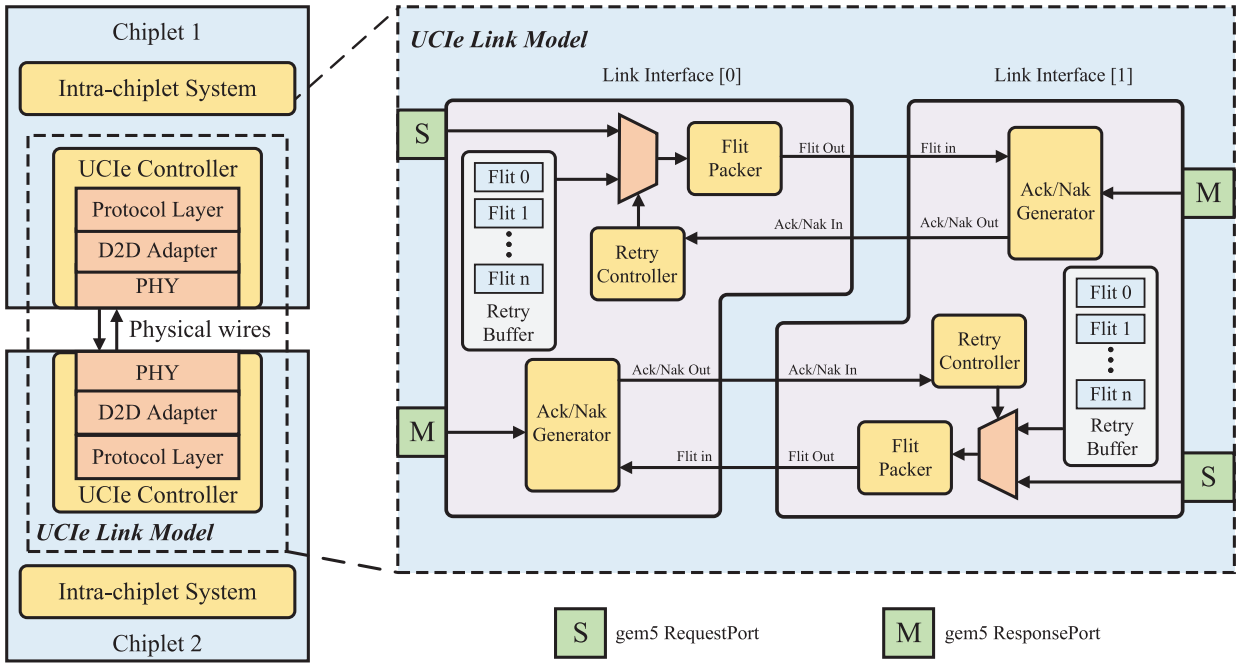



Fig. 2. Overview of gem5 UCIe Link Model.

responses, while the secondary port is responsible for receiving requests and sending responses. To model bidirectional communication via the UCIe link effectively, two separate gem5 links are essential, enabling comprehensive data exchange simulation in both directions.

Figure 2 provides an overview of the UCIe link model. On the left side of the illustration, the model's scope is depicted within a real chiplet-based system, highlighting the communication link between UCIe controller 1 and controller 2 through physical wires. The UCIe controller in this model encompasses the protocol layer, die-to-die adapter, and physical layer (PHY). On the right side of Figure 2, a detailed implementation diagram of the model is presented. The UCIe link model comprises two link interfaces, each containing primary and secondary ports. These link interfaces can be connected to any chiplet component within gem5. Whenever any port receives a gem5 packet, the internal flit packer efficiently packages the Transaction Layer Packet (TLP) into the nearest available flit(s) and transmits it to the peer link interface.

The additional data path in the link model is designed to support the retry mechanism, where flit sequence numbers are represented by timestamps. Within this framework, the flit packager enqueues a copy of the packaged flit packet into the retry buffer. Upon receiving an ACK, the flit packet corresponding to the acknowledged sequence number and all preceding flit packets are released from the retry buffer. Conversely, when an NAK is received, the link interface prevents the reception of additional TLPs and retransmits the flit packet from the retry buffer via a pop operation.

### B. Gem5 Event Schedule of Flit Mode

In the UCIe protocol, data packets are transmitted as flit packets, and the gem5 simulation implementation follows this transmission method. Figure 3 illustrates the encapsulation

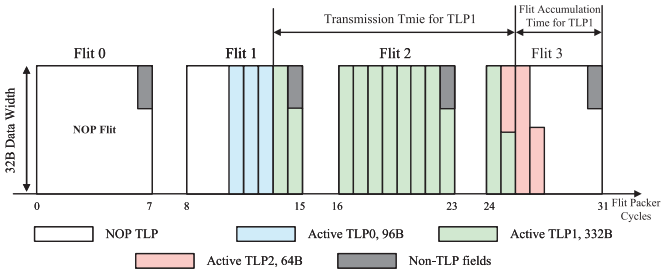



Fig. 3. Simulation results for the networkPacking Process of Two Consecutive Active TLPs.

process of three consecutive active TLPs in flit mode. In Figure 3, the horizontal axis represents time, quantified by the cycle period of the flit packager. In contrast, the vertical axis indicates the processing of 32B (256-bit) broad data within a single cycle. In this scenario, the second active TLP (TLP1), which has a size of 332B, begins its transmission at cycle 14, spans three flits, and occupies 20B of the 32B data width at cycle 25. This scenario simulates complex chiplet communication cases, such as placing TLPs at any DWORD (32-bit) aligned position, the coexistence of multiple TLPs in a single-cycle data path, and the transmission of a single TLP across multiple flits. It is worth noting that the subsequent explanations of event scheduling and retry mechanisms are based on the same scenario depicted in Figure 3.

Additionally, the top of Figure 3 illustrates the latency composition of TLP1 in the UCIe link model (excluding PHY latency). The total latency consists of transmission time and flit accumulation time, representing the time interval from when a valid data packet is encapsulated and sent on the bus of Chiplet 1 to when it is decapsulated and received on Chiplet 2. Due to the pipelined design of the transmitting end, in the absence of a retry mechanism, the protocol layer

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

of the transmitter performs flit encapsulation simultaneously while the PHY serially transmits data at high speed. Therefore, without PHY latency, the transmission time represents both the transmission duration at the sender and the reception duration at the receiver. Meanwhile, the flit accumulation time denotes the time required by the receiver to process the last data bit of a TLP until flit CRC verification is completed.

The latency of the UCIe link is modeled using event-driven programming in gem5. Specifically, when an event is scheduled, it is queued in the event queue corresponding to the specified simulation time. Gem5 employs a global clock object to track and simulate time progression. During each clock cycle in the simulation phase, gem5 schedules events based on priority and executes the associated callback functions. The simulator then advances to the next time step and repeats the event scheduling process only when the event queue is exhausted.

The proposed flit packaging mechanism is an optimization over the previously modeled packaging strategy [14]. In the original scheduling approach, a polling-like event handling method was used—specifically, the packNop() event was scheduled every cycle to check for pending data packets. If a packet was present, the event proceeded to the packet handling routine; if not, the event would exit and be rescheduled in the next clock cycle. This type of event scheduling triggered processing on every clock cycle regardless of whether actual data exchange occurred, which significantly reduced simulation efficiency.

The proposed flit encapsulation mechanism consists primarily of scheduling two tasks: packTlp and sendFlit. The packTlp task is scheduled when a data packet is received. The packTlp handler enqueues the data packet into txQueue after a transmission time, which is calculated based on the size of the TLP packet and the available space in the current flit. Subsequently, at an 8-cycle-aligned time boundary, the FlitSender dequeues and schedules the data packet. In cases where txQueue.push and sendFlit are scheduled simultaneously, and task prioritization is required to ensure successful packet transmission. To efficiently respond to recvPkt() within the same time step, the packTlp task is assigned the highest priority. The sendFlit task is scheduled every eight cycles, and its corresponding handler function iterates through txQueue to transmit data packets to the peer link interface. When there are no active TLPs, the FlitPacker continues to transmit empty data packets while recording relevant metadata, ensuring that incoming active TLPs are placed in the appropriate flit packet locations when they arrive.

Figure 4 illustrates the time sequence of event scheduling for the three consecutive TLPs presented in Figure 3. The task scheduling of FlitPacker and FlitSender occurs in parallel, with arrowed lines representing their scheduling relationships. A total of four flit packets are transmitted in the figure, corresponding to the transmission of three TLP packets. The first flit packet contains no data, serving as an empty packet transmission. In the second flit packet, no TLP0 data is available in the first half, resulting in empty content. When RecvPkt() receives the TLP0 data, the PackTlp task encapsulates the received data into a flit packet. By the eighth

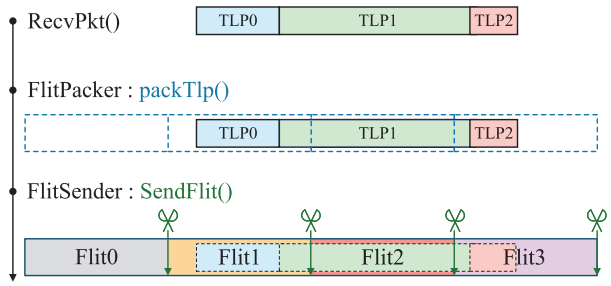



Fig. 4. Sequence Diagram of Event Scheduling.

cycle of this flit, it reaches full capacity and is subsequently transmitted, while the remaining TLP1 data is encapsulated in the third flit packet. The third flit packet continues to transmit TLP1 data until FLIT3 completes the full transmission of TLP1. FLIT3 also encapsulates the TLP2 packet, and after all TLP packets are fully encapsulated, it sends a partial empty packet at the end.

### C. Modeling of Retry Mechanism

The UCIe protocol provides multiple mechanisms to ensure reliable transmission over the link. For example, a cyclic redundancy check (CRC) detects transmission errors, while a retry mechanism corrects errors. The UCIe link model proposed in this paper supports the retry mechanism by utilizing Ack/Nak data link layer packets (DLLP) at the flit granularity.

In UCIe, the Ack/Nak DLLP mechanism operates as follows: when the flit packer assembles 236B of data from the protocol layer, it inserts an 8-bit flit sequence number into the corresponding header field of the flit packet. Although the sequence number is theoretically incremental, in this model, it is instead based on timestamp values. Before transmitting the packaged flit to the peer link interface, a copy of the flit packet is stored in the retry buffer. The retry buffer retains the flit until an Ack DLLP is received, confirming its error-free reception. Conversely, if the receiver detects a corrupt flit, it sends a Nak DLLP, which includes the sequence number of the last correctly received flit. Upon receiving the Nak DLLP, the sender removes and releases all flits in the retry buffer with sequence numbers less than or equal to the one specified in the Nak packet and retransmits all stored flits from the buffer.

During the implementation process, it should be noted that the sender can choose to skip NOP flits in replay, the sequence number '0' is specially used for Idle flits, and the NOP flits do not consume the flit sequence number. This Ack/Nak DLLP mechanism ensures reliable packet transmission, guaranteeing that data frames are correctly received and that retransmissions are triggered in case of errors. While this mechanism effectively addresses bit error rate (BER) issues in data transmission, it still has limitations when network congestion occurs. In the original retransmission mechanism [14], the ACK/NAK DLLP-based retry strategy relied solely on the received packets to determine their correctness and issue corresponding responses to indicate whether retransmission was needed. However, when the receiver was busy and unable to promptly send ACK/NAK responses, this could lead to an

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

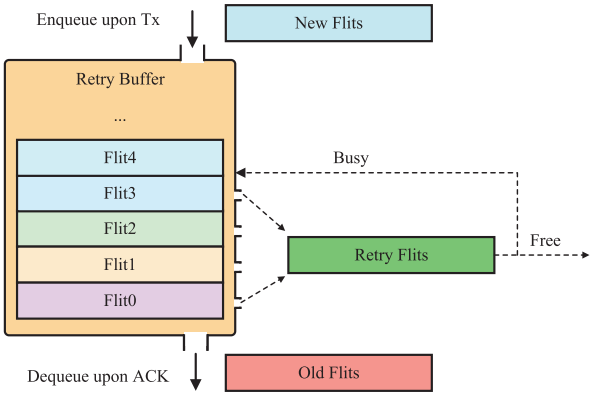



Fig. 5. Schematic Diagram of the Retry Buffer and Flit Descriptor.

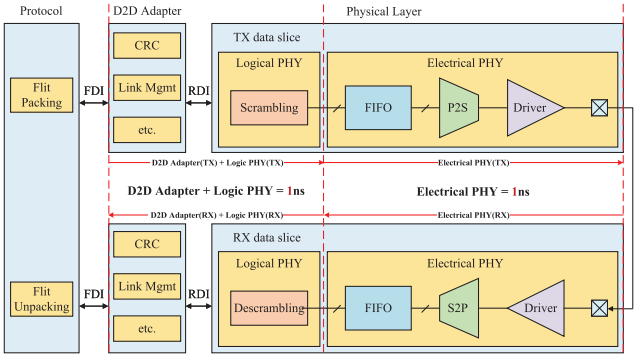



Fig. 6. Latency Illustration of UCIe Link Behavioral-Level Model.

unnecessary increase in the number of retransmitted packets, and in some cases, even cause simulation stalls. To address this issue, the UCIe link model proposed in this paper introduces a single retry mechanism to handle transmission failures caused by network congestion. The proposed retransmission scheme is illustrated in Figure 5.

The proposed link retransmission scheme introduces a single retry mechanism on top of the existing framework, ensuring enhanced data tracking for each flit packet after transmission. When the link is congested, the data packet cannot be received, and continuously attempting transmission may lead to a deadlock. To prevent this, the transmission of the flit packet is temporarily paused, and the packet is stored in the Retry Buffer, awaiting retransmission once the data link becomes available. Additionally, if the sender does not receive acknowledgment feedback from the receiver within a predefined time frame, the receiver will automatically retransmit the data. Once the data packet is successfully received, it is then processed through the ACK/NAK DLLP mechanism for further validation.

The proposed retransmission scheme retains the advantages of the original Ack/Nak DLLP retry mechanism while effectively addressing slow transmission and deadlock issues. When the model detects link congestion, it temporarily stores unsuccessfully transmitted packets and retransmits them once the link becomes idle, ensuring that packets are correctly received by the destination. As an extension and enhancement of the Ack/Nak DLLP retry mechanism, this approach effectively reduces simulation stalls, improves simulation performance, and has been validated through experiments to ensure its reliability and correctness.

### *D. Modeling of Latency Mechanism*

In the design of the UCIe latency model, this work constructs the framework based on the key performance metrics and reference architectures stipulated in UCIe [16], as depicted in Figure 6. The computation of latency references the model work on SerDes delay by Butko set al. [17], especially the latency from the FDI interface to the physical pins. In the UCIe protocol, the latency defined under key performance targets includes the combined latency of the adapter and PHY layers, specifically covering the delay from the FDI interface to the physical pins. Studies have shown that the total latency consists of four main components: adapter latency, logical PHY latency, electrical PHY latency, and channel propagation delay. The first three types of latency are primarily dependent on the semiconductor process technology. Circuits fabricated with more advanced nodes operate at higher clock frequencies and therefore exhibit lower latency. Channel propagation delay, on the other hand, is closely related to the packaging technology. Factors such as interconnect length, bump pitch and size, and the number of through-silicon vias (TSVs) in different packaging methods all affect latency [18].

To accommodate variations in manufacturing processes while meeting the overall latency target defined in the UCIe specification, we developed a parameterized configuration architecture for total latency. This model supports parametrized PHY configurations, allowing dynamic compensation for process-induced variations. To maintain consistency with the design baseline, we adopt the 2ns latency reference defined by the UCIe 1.0 specification. According to official data provided by the UCIe consortium [16], this 2ns total delay consists of 1ns for the adapter and TRX logical PHY, and 1ns for the TRX electrical PHY. The delay introduced by standard or advanced packaging technologies is also treated as an independently configurable parameter within the overall latency, enabling precise calibration of the model's configurable delay. The frequency and bit-width parameters of the protocol layer and adapter layer in the model are matched with the processing speed of 64GT/s in the electrical PHY under a single module, standard packaging.

## IV. Experimental Evaluations

To verify and evaluate the latency and functional correctness of the proposed UCIe link model, this work conducts experimental simulations. Two topologies are used in the simulations. The first topology simulates a data transmission and reception scenario between two chiplets, primarily for validating the latency accuracy of the UCIe link model. The second topology represents a general computing scenario, where a CPU chiplet and an accelerator chiplet are interconnected using the UCIe link model, with each chiplet having its own memory. This experiment compares the overall system performance under three different configurations: the UCIe

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

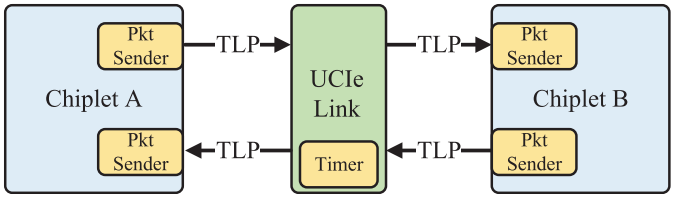



Fig. 7. Dual-Chiplet Transmission and Reception Test Diagram.

TABLE I
LATENCY MEASUREMENT

| Date Size(B) | TLP Size(B) | Theoretical Latency(ns) | Simulated Latency(ns) |
|---|---|---|---|
| 16 | 32 | 18 | 18.1093 |
| 48 | 64 | 22 | 21.8926 |
| 80 | 96 | 26 | 26.1083 |
| 112 | 128 | 30 | 31.8254 |
| 240 | 256 | 46 | 46.1163 |
| 496 | 512 | 78 | 77.8503 |
| 880 | 896 | 126 | 126.1064 |
| 1008 | 1024 | 142 | 141.9290 |
| 2032 | 2048 | 270 | 270.0851 |
| 4080 | 4096 | 526 | 525.9462 |

link model, the PCIe model, and direct interconnection. The simulation experiments were conducted on a system with an Intel(R) Xeon(R) Silver 4214R CPU running Ubuntu 18.04.6 LTS. The experimental setup utilized gem5 version 21.2.1.0, and the memory specification in the simulation was 4 GB. The following sections provide a detailed explanation of the experiments conducted for each topology.

### A. Latency Simulation of UCIe Link Model

To effectively evaluate the latency consistency between the modeled UCIe protocol and the actual UCIe protocol, this work employs a dual-chiplet architecture, as illustrated in Figure 7. The two chiplets continuously transmit and receive test data packets of predefined sizes, which are selected based on the work by Sharma [19] and serve as inputs for system response evaluation. A timer unit within the UCIe link is used to record the time elapsed during data packet transmission and reception. Table I provides a detailed breakdown of the calculated latency, excluding any retransmissions, while Equation 1 defines the latency calculation using the average accumulation time [14]. This accumulation time is measured from the reception of the last byte of a TLP to the completion of flit packet processing at the receiver, influenced by the CRC mechanism. Theoretically, this accumulation time depends on the TLP size and the operating frequency of the data path. For a data path operating at f GHz, the accumulation time value is expressed as nT ns, where n ranges from 0 to (32f - 1).

$$\begin{aligned}\hat{t}_{accumulation}(ns) &= \frac{\sum_{n=0}^{32f-1} nT}{32f} = 16 - \frac{1}{2f}\\ t_{latency}(ns) &= t_{transmission} + \hat{t}_{accumulation}\\ &= \frac{TLPSize(B) \times 8}{64\text{Gb/s}} + 16 - \frac{1}{2f}\end{aligned} \tag{1}$$

To reduce experimental randomness, the latency of the link model is derived from the average of 1,000,000 simulation iterations. In the test configuration, the Pkt Sender module of a single chiplet initiates a memory write operation through the sendTimingReq function. This function utilizes the gem5 TimingRequest object to replicate the memory operation at the protocol layer. Latency tracking begins when the secondary port of Link Interface 0 receives the TimingReq, and it ends when the primary port of Link Interface 1 schedules the corresponding TimingResp. This method enables precise measurement of data transmission latency through the UCIe link, excluding operating system overhead.

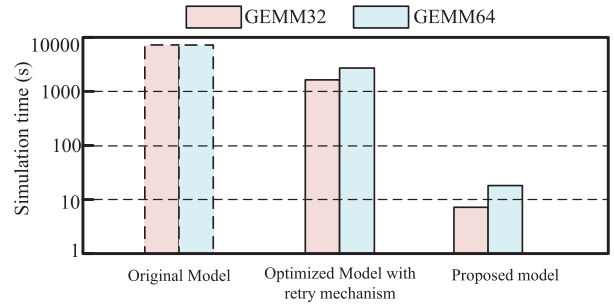



Fig. 8. Optimization Measurement.

The TLP size in Table I is 16B larger than the data size as the TLP needs to contain 16B headers. As shown in Table I, the latency of the modeled UCIe Link closely matches the theoretical latency, along with the difference between them is less than 0.3ns at most. When the TLP size is 32B, the difference between the simulated and theoretical latency is the largest, with a maximum deviation of 0.5% from the theoretical value. However, when the TLP size reaches 4096B, the UCIe Link model latency is nearly identical to the theoretical latency. The initial deviation is primarily due to system instability at the start of the simulation. Once the simulation stabilizes, the latency values become highly consistent. Therefore, the proposed UCIe Link model provides an accurate evaluation of link latency.

This experiment also demonstrates the results of optimizing the UCIe protocol within the gem5 environment. The proposed UCIe Link model has been specifically optimized for gem5 simulation. The optimization mainly includes the following: During simulation, only the protocol layer needs to be implemented, while the physical layer and adapter layer, which are required in the actual protocol, can be left unmodeled. As a result, the total flit size is set to 236B. Since gem5 operates with an event-driven simulation mechanism, the implementation is designed to minimize the number of scheduled events while ensuring efficient UCIe protocol transmission. In the physical implementation of UCIe, the link must remain active even when no data exchange occurs, ensuring readiness for data transmission at any time. However, this behavior is unnecessary in the simulation, allowing it to be omitted for efficiency. The comparison results before and after optimization are shown in Figure 8.

As shown in Figure 8, in the original UCIe link protocol, standard GEMM programs cannot be simulated due to issues such as deadlocks and congestion. Therefore, dashed lines are used to represent this. After incorporating the proposed

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

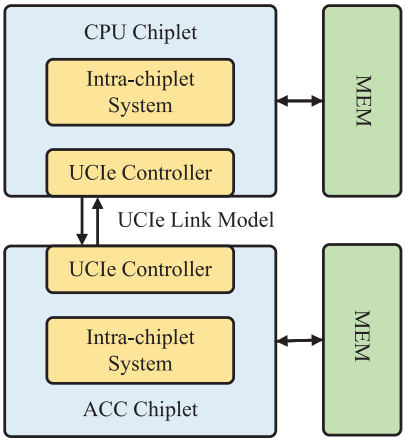



Fig. 9. CPU-Accelerator System Test Diagram.

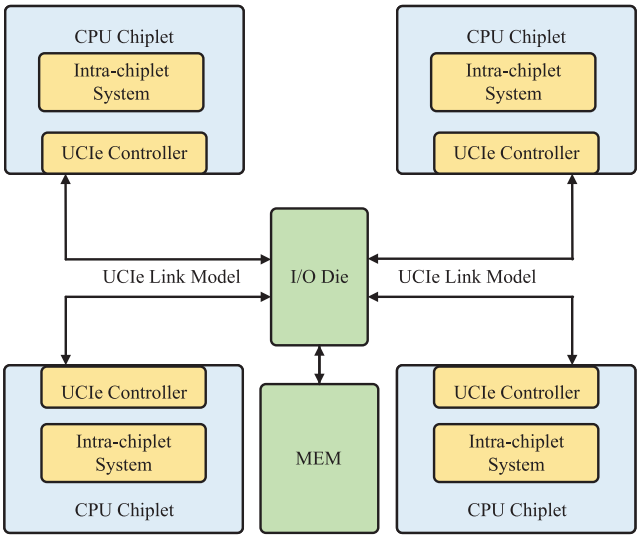



Fig. 10. Multi-chiplets System Test Diagram.

retransmission mechanism optimizations, the original UCIe link protocol can successfully simulate these cases, though with relatively slow simulation speed. With further logic optimizations, the enhanced UCIe link model is able to produce simulation results for the GEMM programs within tens of seconds. Compared to the original UCIe link protocol, the optimized link model has achieved a significant improvement in simulation speed, realizing an over 240× acceleration. This enhancement makes the link model practically applicable for real-world use.

### *B. General Computing Scenario Simulation of UCIe Link Model*

In this section, a heterogeneous computing environment consisting of a CPU chiplet and an accelerator chiplet is established using the gem5 simulator, with interconnection implemented via the UCIe protocol.

The system architectures used in this experiment are illustrated in Figures 9 and 10. Figure 9 presents a complete system structure comprising a CPU chiplet, an accelerator chiplet, and memory. In this system, data communication between the CPU chiplet and the accelerator chiplet utilizes various data transmission methods, while communication between each chiplet and its respective memory follows the chiplet's original communication protocol. Figure 10 depicts a full multi-chiplet system architecture, which consists of four CPU chiplets connected to a single IO die. In both system architectures, the memory selection can be either DDR or HBM, depending on the configuration. The accelerator design in this work is based on the implementation by Vieira et al. [20]. The experiment compares three different data transmission methods:

UCIe Interconnect: data is transmitted between the CPU and accelerator chiplet via the UCIe link. PCIe Interconnect: data is transmitted between the CPU and accelerator chiplet via the PCIe link. Direct Data Transfer: the default gem5 data transmission path is used, bypassing the UCIe adapter layer for direct data exchange. To ensure consistent testing conditions, all experiments are conducted using the UCIe 256B flit mode based on PCIe 6.0.

To comprehensively assess the performance overhead of UCIe in different computational scenarios, the following representative computational tasks are selected for evaluation: Matrix Multiplication: widely used in deep learning and scientific computing. Convolution and Transposed Convolution Computation: a core operation in neural network inference. Fast Fourier Transform (FFT): a fundamental operator in signal processing, and so on. All test tasks are executed under the same computational scale to ensure fair comparison. In our model, each UCIe transmission is counted, and once the number of transmissions reaches the threshold derived from the configured bit error rate, a bit error is considered to have occurred. The bit error rate (BER) in the experiment was configured to $10^{-10}$.

The experimental results are presented in Table II. In this work, three memory models (DDR3, DDR4, and HBM) from DRAMSim were utilized. Each model is associated with its respective configuration file, which defines simulation parameters and options, including DRAM architecture, timing characteristics, power consumption, and other relevant details. The key performance parameters are presented in Table III.

The experiment shows that UCIe interconnect incurs approximately a 3% performance loss compared to direct data transfer when using DDR as memory. Specifically, in the Conv task, the performance loss is approximately 2.7%, primarily due to the additional protocol overhead in data transmission. In the MaxPool and ReLU tasks, the UCIe transmission method incurs a 2.6% performance loss compared to the direct data transfer method. In the GEMM task, the computation time of UCIe interconnect is 2.3% longer than that of direct data transfer. In the FFT task, the UCIe approach is 0.9% slower than direct data transfer. Different tasks exhibit varying overall performance differences due to their differing degrees of data dependency.

The experiment also compares the overall system performance of UCIe transmission, PCIe transmission, and direct transmission under two memory configurations: DDR3 and

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYSTEM PERFORMANCE UNDER DIFFERENT CONDITIONS

| Method | Memory type | Conv2d | Conv3d | MaxPool | ReLU | GEMM | FFT | DeConv2d | Multi-chiplet |
|---|---|---|---|---|---|---|---|---|---|
| UCIe Link Model | DDR3[1] | 19674 | 17319 | 12841 | 7937 | 404779 | 396 | 1541628 | 60571 |
| | DDR4[2] | 19481 | 17080 | 12835 | 7928 | 390654 | 396 | 1469870 | 59617 |
| | HBM[3] | 22132 | 19618 | 14629 | 8862 | 454862 | 416 | 1032510 | 64155 |
| PCIe6.0 Link Model | DDR3 | 21908 | 19236 | 14361 | 8808 | 448774 | 415 | 1626146 | 64485 |
| | DDR4 | 21791 | 19125 | 14277 | 8806 | 435002 | 414 | 1557328 | 63281 |
| | HBM | 24557 | 21953 | 16105 | 9757 | 498087 | 435 | 1128786 | 69464 |
| PCIe5.0 Link Model | DDR3 | 24363 | 21412 | 15810 | 9746 | 496562 | 435 | 1718748 | 70699 |
| | DDR4 | 24236 | 21292 | 15733 | 9743 | 431631 | 433 | 1649194 | 69016 |
| | HBM | 26872 | 23704 | 17759 | 10658 | 545866 | 455 | 1215508 | 81113 |
| Directly Linked | DDR3 | 19134 | 16810 | 12523 | 7741 | 395797 | 393 | 1521050 | 60111 |
| | DDR4 | 19006 | 16652 | 12465 | 7731 | 381556 | 392 | 1452232 | 59487 |
| | HBM | 21978 | 19254 | 14397 | 8668 | 446868 | 412 | 1031040 | 63972 |
| Acceleration Ratio(UCIe Compared with PCIE6.0) | DDR3 | 0.898 (0.000077)[1] | 0.900 (0.000545) | 0.894 (0.000620) | 0.901 (0.000258) | 0.902 (0.000460) | 0.954 (0.000513) | 0.948 (0.000361) | 0.939 (0.000375) |
| | DDR4 | 0.894 (0.000110) | 0.893 (0.000195) | 0.899 (0.000862) | 0.900 (0.000153) | 0.898 (0.000843) | 0.957 (0.000563) | 0.944 (0.000383) | 0.942 (0.000330) |
| | HBM | 0.901 (0.000023) | 0.894 (0.000175) | 0.908 (0.000703) | 0.908 (0.000185) | 0.913 (0.000905) | 0.956 (0.000538) | 0.915 (0.000867) | 0.924 (0.000452) |
| Acceleration Ratio(UCIe Compared with Directly Linked) | DDR3 | 1.028 (0.000033) | 1.030 (0.000211) | 1.025 (0.000059) | 1.025 (0.000577) | 1.023 (0.000314) | **1.008** (0.000752) | 1.013 (0.000186) | 1.008 (0.000154) |
| | DDR4 | 1.025 (0.000046) | 1.026 (0.000151) | 1.030 (0.000157) | 1.025 (0.000144) | 1.024 (0.000005) | 1.010 (0.000905) | 1.012 (0.000067) | **1.002** (0.000090) |
| | HBM | **1.007** (0.000065) | **1.019** (0.000318) | **1.016** (0.000778) | **1.022** (0.000216) | **1.018** (0.000493) | 1.010 (0.000401) | **1.001** (0.000292) | 1.003 (0.000391) |

[1] The DDR3 used in this test is DDR3_1600_8x8().
[2] The DDR4 used in this test is DDR4_2400_16x4().
[3] The HBM used in this test is HBM_2000_4H_1x64().
[4] The standard deviation is shown in parentheses.

TABLE III
MEMORY COMPONENT SPECIFICATIONS

| Type | Banks | tCK(ns) | tRCD(ns) | Data Width(bit) |
|---|---|---|---|---|
| DDR3 | 8 | 1.25 | 13.75 | 64 |
| DDR4 | 8 | 0.833 | 14.16 | 64 |
| HBM | 16 | 1 | 12 | 1024 |

DDR4. As shown in Table II, replacing DDR3 with DDR4 results in an overall system performance improvement. However, the performance gap between the three transmission methods remains relatively small.

In certain cases, using HBM as the memory results in a decrease in system performance compared to using DDR. This is because HBM is better suited for applications involving high-bandwidth, large data transfers, and its transmission frequency is generally lower than that of DDR. In cases where the high bandwidth of HBM cannot be fully utilized, its advantages are not realized. In this scenario, the overall performance of the system using HBM as memory is lower compared to a system using DDR as memory. To address this problem, we have also designed dedicated test cases for HBM: DeConv2d, where the input data volume is relatively small, while the accelerator has a high computational workload and significant data interaction with memory. In these cases, the limitation from cpu's performence is minimized, and the accelerator's data access width is expanded to 512 bits to reflect the synergy with the UCIe protocol. At the same point, we used HBM with 8 physical channels, each containing 2 pseudo-channels, resulting in a total data bandwidth of 1024 bits. At this point, the high bandwidth advantage of HBM is fully utilized, achieving up to a 1.4× speedup compared to DDR4. And the overall system performance difference between using UCIe and direct connection is minimal, with only a 0.1% gap, which demonstrates the synergy between HBM and UCIe. On the other hand, even when CPU data transfer becomes the bottleneck, the system with HBM as memory exhibits the smallest performance gap with UCIe protocol and direct connection, with a minimal performance loss of only 0.7%. When comparing UCIe and PCIe protocols, the performance difference is also the smallest when using HBM as memory. This is due to HBM's high parallel access capability, which naturally adapts to UCIe and PCIe protocols that feature multiple parallel data transmission channels, leading to performance close to that of direct connection. Therefore, we conclude that the selection of memory units should consider the specific application scenario to achieve optimal cooperation between the memory and chiplet communication protocols. This provides a reference for future chiplet-based designs.

To test the multi-chiplet system, we also conducted tests on a multi-chiplet system as shown in Figure 10, with results presented in the "multi-chiplet" column of Table II. It can be observed that, in the multi-chiplet system, the performance of the proposed UCIe protocol is similar to that of the dual-chiplet system, indicating that the UCIe protocol has scalability and can support modeling and simulation of multi-chiplet systems.

To ensure the stability of the acceleration ratio data, we performed statistical analysis on these data and calculated the standard deviation for each group. From the standard deviation values in Table II, it is evident that the largest standard deviation does not exceed 0.001, indicating that the acceleration ratio data is reliable.

The experiment confirms that UCIe interconnect introduces some protocol overhead compared to direct data transfer, but this overhead remains within an acceptable range (below

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.

3%). The primary sources of this overhead include Protocol encapsulation overhead: The UCIe protocol enforces specific data encapsulation requirements, resulting in an additional transmission burden. Link flow control mechanisms: Unlike direct data transfer, UCIe interconnect requires additional handshaking and flow control mechanisms to ensure reliable data transmission, which may introduce some throughput loss.

At the same time, we also observe that the overall performance of the model utilizing the UCIe link is higher than that of the model using the PCIe protocol. This is primarily because UCIe is specifically designed for high-speed interconnects between chiplets, allowing for direct die-to-die (D2D) communication, whereas PCIe requires data transmission through external components such as motherboards and cables. The shorter transmission path and wider data channels in UCIe contribute to its superior performance compared to PCIe-based systems. By replacing PCIe with UCIe, the overall system performance can be improved by approximately 10%.

Despite the additional overhead of UCIe compared to direct data transfer, its modular design and standardized interconnect provide significant advantages in multi-chiplet interconnects and large-scale chip integration. UCIe interconnect offers more excellent compatibility and scalability, making it highly beneficial for heterogeneous computing environments. The simulation of UCIe links enables low-cost system performance evaluation, providing valuable insights for future optimizations and significantly reducing development costs for large-scale chiplet interconnect applications.

## V. Conclusion and Future Work

In this paper, we propose an improved UCIe link model based on gem5. The proposed model incorporates optimized flit packing logic, enhances gem5's event scheduling mechanism, and refines the retry mechanism. Experimental results show that the new model achieves up to a 240× improvement in simulation speed compared to the original model, while the measured link latency deviates by only 0.5% from theoretical predictions. We evaluate and compare the system-wide performance of the proposed UCIe link model, the PCIe model, and direct interconnects under different workloads in a general CPU-accelerator chiplet system architecture. The performance difference between the UCIe model and direct interconnect is as low as 0.7% in the best-case scenario. Our proposed model provides guidance and reference for future chiplet system design.

## References


[1] G. H. Loh, S. Naffziger, and K. Lepak, "Understanding chiplets today to anticipate future integration opportunities and limits," in *Proc. Design, Autom. Test Europe Conf. Exhib. (DATE)*, 2021, pp. 142–145.

[2] A. Das, M. Palesi, J. Kim, and P. Pratim Pande, "Chip and package-scale interconnects for general-purpose, domain-specific, and quantum computing systems—Overview, challenges, and opportunities," *IEEE J. Emerg. Sel. Topics Circuits Syst.*, vol. 14, no. 3, pp. 354–370, Sep. 2024.

[3] D. Das Sharma, G. Pasdast, Z. Qian, and K. Aygun, "Universal chiplet interconnect express (UCIe): An open industry standard for innovations with chiplets at package level," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 12, no. 9, pp. 1423–1431, Sep. 2022.

[4] B. Wang, G. Chen, L. Y. Foo, and X. Jiang, "Signal and power integrity design of advanced interface bus (AIB) for FPGA packages," in *Proc. IEEE 32nd Conf. Electr. Perform. Electron. Packag. Syst. (EPEPS)*, Oct. 2023, pp. 1–3.

[5] N. Binkert et al., "The gem5 simulator," *ACM SIGARCH Comput. Archit. news*, vol. 39, no. 2, pp. 1–7, 2011.

[6] J. Lowe-Power et al., "The gem5 simulator: Version 20.0+," 2020, *arXiv:2007.03152*.

[7] Y. Li, J. Zhang, F. Wu, Y. Xie, J. Pang, and G. Li, "A UCIe interconnect channel design up to 32Gbps on 2.5D Si-interposer," in *Proc. 25th Int. Conf. Electron. Packag. Technol. (ICEPT)*, Aug. 2024, pp. 1–3.

[8] S. Shimomura, Y. Uematsu, K. Kikuchi, H. Shimamoto, Y. Araga, and S. Ouchi, "Single-layer wiring design in UCIe to realize low-cost interposer substrate," in *Proc. IEEE 33rd Conf. Electr. Perform. Electron. Packag. Syst. (EPEPS)*, Oct. 2024, pp. 1–3.

[9] J. Kim et al., "Effective interface simulation approach based on peak distortion analysis for UCIe IPs," in *Proc. IEEE 74th Electron. Compon. Technol. Conf. (ECTC)*, May 2024, pp. 1104–1107.

[10] M. Alian, K. P. Srinivasan, and N. S. Kim, "Simulating PCI-express interconnect for future system exploration," in *Proc. IEEE Int. Symp. Workload Characterization (IISWC)*, Sep. 2018, pp. 168–178.

[11] R. Medina et al., "System-level exploration of in-package wireless communication for multi-chiplet platforms," in *Proc. 28th Asia South Pacific Design Autom. Conf.*, Jan. 2023, pp. 561–566.

[12] T. Jose and D. Shankar, "Performance modeling of a heterogeneous computing system based on the UCIe interconnect architecture," in *Proc. IEEE Space Comput. Conf. (SCC)*, Jul. 2023, pp. 5–10.

[13] H. Zheng, K. Wang, and A. Louri, "A versatile and flexible chiplet-based system design for heterogeneous manycore architectures," in *Proc. 57th ACM/IEEE Design Autom. Conf. (DAC)*, Jul. 2020, pp. 1–6.

[14] X. Li et al., "MUG5: Modeling of universal chiplet interconnect express (UCIe) standard based on gem5," in *Proc. IEEE 15th Int. Conf. ASIC (ASICON)*, Oct. 2023, pp. 1–4.

[15] D. D. Sharma, "PCI express 6.0 specification at 64.0 GT/s with PAM-4 signaling: A low latency, high bandwidth, high reliability and cost-effective interconnect," in *Proc. IEEE Symp. High-Perform. Interconnects (HOTI)*, Aug. 2020, pp. 1–8.

[16] D. D. Sharma, "System on a package innovations with universal chiplet interconnect express (UCIe) interconnect," *IEEE Micro*, vol. 43, no. 2, pp. 76–85, Mar. 2023.

[17] A. Butko, R. Garibotti, L. Ost, and G. Sassatelli, "Accuracy evaluation of GEM5 simulator system," in *Proc. 7th Int. Workshop Reconfigurable Commun.-Centric Syst.-Chip (ReCoSoC)*, Jul. 2012, pp. 1–7.

[18] A. Li et al., "A method to improve 3D interconnections resource utilization and reliability in hybrid bonding process considering the effects on signal integrity," in *Proc. IEEE 73rd Electron. Compon. Technol. Conf. (ECTC)*, May 2023, pp. 2131–2136.

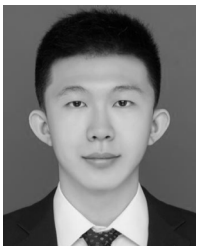
[19] D. D. Sharma. (2022). *Pcie 6.0 Specification: A High-performance I/o Interconnect for Advanced Networking Applications*. [Online]. Available: https://www.openfabrics.org/wp-content/uploads/2022-workshop/2022-workshop-presentations/206_DDasSharma.pdf

[20] J. Vieira, N. Roma, G. Falcao, and P. Tomas, "Gem5-ndp: Near-data processing architecture simulation from low level caches to DRAM," in *Proc. IEEE 34th Int. Symp. Comput. Archit. High Perform. Comput. (SBAC-PAD)*, Nov. 2022, pp. 41–50.

**Kunyue Li** received the B.E. degree in microelectronics science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2022, where he is currently pursuing the Ph.D. degree.

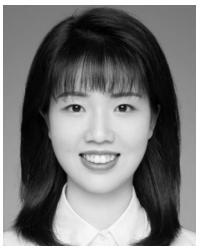
His current research interests include chiplet architecture design and network-on-chip.

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.



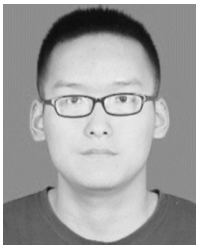
**Shuaipeng Li** received the B.E. degree in microelectronics science and engineering from Harbin Institute of Technology, Harbin, China, in 2021. He is currently pursuing the Ph.D. degree with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

His current research interests include system modeling and high-performance physical design of chiplet systems.



**Xiaoyan Li** received the B.E. and M.S. degrees in microelectronics science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2021 and 2024, respectively.

She is currently an Engineer in AMD. Her current research interests include inter-chip protocol and AI tools.



**Zizheng Dong** received the B.E. degree in microelectronics science and engineering from Lanzhou University, Lanzhou, China, in 2019. He is currently pursuing the Ph.D. degree with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University, Shanghai, China.

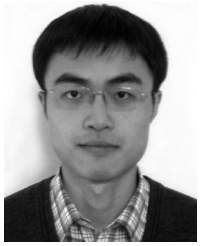
His current research interests include high-speed on-chip interconnect and high-performance physical design of 3-D ICs.

**Sai Gao** received the B.E. degree in electronic science and technology from Xidian University, Xi'an, China, in 2018, and the M.S. degree in electronic science and technology from Shanghai Jiao Tong University, Shanghai, China, in 2021, where he is currently pursuing the Ph.D. degree with the Department of Micro-Nano Electronics.

His current research interests include high-performance physical design of chiplet systems and high-speed on-chip interconnect.

**Jialei Sun** received the B.E. degree in microelectronics science and engineering from Shanghai Jiao Tong University, Shanghai, China, in 2023, where he is currently pursuing the master's degree with the Department of Micro-Nano Electronics.

His current research interests include FPGA acceleration and heterogeneous acceleration systems.



**Naifeng Jing** (Senior Member, IEEE) received the B.E., M.S., and Ph.D. degrees from the School of Electronic, Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai, China, in 2005, 2008, and 2012, respectively.

He is currently a Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University. His research interests include high-performance and high reliability computing architecture, emerging in-memory computing architecture, and their related hardware/software co-design.

**Qin Wang** (Member, IEEE) received the B.S. degree from the University of Electronics Science and Technology of China, Chengdu, China, in 1997, and the Ph.D. degree from Shanghai Jiao Tong University, Shanghai, China, in 2004.

She is currently a Professor with the Department of Micro-Nano Electronics, Shanghai Jiao Tong University. Her research interests include high performance processor and in-memory computing.

**Guanghui He** (Member, IEEE) received the B.S. degree in electronic engineering from the University of Electronic Science and Technology of China, Chengdu, China, in 2002, and the Ph.D. degree in electronic engineering from Tsinghua University, Beijing, China, in 2007.

He is currently a Professor with the Department of Micro/Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University, Shanghai, China. He is a Professor with the Department of Micro/Nano Electronics and the MoE Key Laboratory of Artificial Intelligence, Shanghai Jiao Tong University. His research interests include energy efficient algorithms and circuits design for wireless communication, image processing, emerging memory devices, and artificial intelligent systems.

**Jianfei Jiang** (Member, IEEE) received the B.S. degree from Zhejiang University, Hangzhou, China, and the M.S. and Ph.D. degrees from Shanghai Jiao Tong University, Shanghai, China. He is currently an Associate Professor with the Department of Microelectronics and Nanoscience, Shanghai Jiao Tong University. His current research interests include low-power circuit design and 2.5D/3D IC design.

Authorized licensed use limited to: Synopsys. Downloaded on February 15,2026 at 15:48:49 UTC from IEEE Xplore. Restrictions apply.